SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE CODE: CSA5112
COURSE NAME: Cryptography and Network Security

COURSE OUTCOME COVERED

CO2: Analyze Public Key crypto system strategies using RSA and key Exchange for Encryption algorithm to strengthen information security. (BL4,BL5)

Analytical Problem Solving: 40%

QUESTIONS: Total Marks: 50 Annexure A

Code of Conduct:

I D.S.THARUNIYA PRIYA (192511317) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: *D.S.Tharuniya Priya*

Annexure A
Analytical Problem Solving

1. RSA Encryption Analysis

Given p=11, q=13

a. Compute n ϕ(n)

b. Choose e=7 and find the private key d

c. Encrypt the message M=9

d. Decrypt the ciphertext and verify the result

e. Analyze how changing e affects security

Answer 1: RSA Encryption Analysis

Given: p = 11, q = 13, public exponent e = 7, and message M = 9.

(a) Calculation of n and φ(n)

The modulus is calculated by multiplying the two prime numbers:

$n = p \times q = 11 \times 13 = 143$.

Euler's totient function for two distinct primes is :

$\varphi(n) = (p - 1)(q - 1)$.

Therefore, $\varphi(n) = (11 - 1)(13 - 1) = 10 \times 12 = 120$.

Hence, n = 143 and φ(n) = 120.

(b) Finding the private key d

The private exponent d is the modular multiplicative inverse of e. It must satisfy $e \times d \equiv 1 \pmod{\varphi(n)}$.

Substituting the values gives $7d \equiv 1 \pmod{120}$.

Using the Extended Euclidean Algorithm:

$120 = 17 \times 7 + 1$.

Rearranging, $1 = 120 - 17 \times 7 = (-17) \times 7 + 120$.

Thus, $d \equiv -17 \pmod{120}$.

Adding 120 gives $d = -17 + 120 = 103$.

Verification: $7 \times 103 = 721 = 6 \times 120 + 1$, so $7 \times 103 \equiv 1 \pmod{120}$.

Public key = (e, n) = (7, 143).

Private key = (d, n) = (103, 143).

(c) Encryption of M = 9

RSA encryption uses $C = M^e \bmod n$.

Therefore, $C = 9^7 \bmod 143$.

Using repeated squaring: $9^2 = 81$, and $9^4 = 81^2 \bmod 143 = 6561 \bmod 143 = 126$.

Since $7 = 4 + 2 + 1$, $9^7 \bmod 143 = (9^4 \times 9^2 \times 9) \bmod 143 = (126 \times 81 \times 9) \bmod 143$.

$126 \times 81 = 10206$, and $10206 \bmod 143 = 53$. Then $53 \times 9 = 477$, and $477 \bmod 143 = 48$.

Therefore, the ciphertext is C = 48.

(d) Decryption and verification

RSA decryption uses $M = C^d \bmod n$.

Substituting the values gives $M = 48^{103} \bmod 143$.

Repeated-squaring values are: $48^1 \bmod 143 = 48$; $48^2 \bmod 143 = 16$; $48^4 \bmod 143 = 113$;

$48^8 \bmod 143 = 42$;

$48^{16} \bmod 143 = 48$;

$48^{32} \bmod 143 = 16$;

$48^{64} \bmod 143 = 113$.

Since $103 = 64 + 32 + 4 + 2 + 1$, $48^{103} \bmod 143 = (48^{64} \times 48^{32} \times 48^4 \times 48^2 \times 48) \bmod 143 = 9$.

The decrypted message is 9, which matches the original message M = 9.

Therefore, encryption and decryption are verified.

(e) Effect of changing e on security

The public exponent e must satisfy $\gcd(e, \varphi(n)) = 1$. If e is changed, the private exponent d must be recalculated as the modular inverse of e modulo $\varphi(n)$.

A small public exponent can make encryption faster. However, if RSA is used without proper padding or under unsuitable conditions, small exponents can contribute to certain attacks. A larger exponent generally increases encryption computation, but a larger e by itself does not automatically make RSA more secure.

The commonly used public exponent e = 65537 offers efficient encryption and avoids some issues associated with very small exponents. Practical RSA security mainly depends on a sufficiently large modulus, secure padding such as RSA-OAEP, correct implementation, and protection of the private key.

Conclusion:

e = 7 is mathematically valid for this example.

However, n = 143 is far too small for real-world security. Real systems use much larger keys and appropriate padding.

2. Block Cipher Mode Evaluation

Given plaintext blocks:

P=[1010,1010,1111,1010], Key = K=1100

a. Encrypt using ECB and CBC modes (Assume IV = 0000) b. Compare the ciphertext outputs

c. Analyze which mode hides patterns better and explain why ?

Answer 2: Block Cipher Mode Evaluation

Given:

P = [1010, 1010, 1111, 1010], key K = 1100, and IV = 0000.

Assumption:

The question does not specify the actual block-cipher algorithm. For this worked example, use the simplified teaching function $E_k(X) = X$ XOR K. These results illustrate the modes only; XOR with a key is not a secure block cipher.

(a) Encryption using ECB mode

In Electronic Codebook (ECB) mode, each plaintext block is encrypted independently:

$C_i = E_k(P_i) = P_i$ XOR K.

Block 1: 1010 XOR 1100 = 0110, so $C_1 = 0110$.

Block 2: 1010 XOR 1100 = 0110, so $C_2 = 0110$.

Block 3: 1111 XOR 1100 = 0011, so $C_3 = 0011$.

Block 4: 1010 XOR 1100 = 0110, so $C_4 = 0110$.

Therefore, ECB ciphertext = [0110, 0110, 0011, 0110].

Encryption using CBC mode (IV = 0000)

In Cipher Block Chaining (CBC), each plaintext block is XORed with the previous ciphertext block before encryption:

$C_i = E_k(P_i \text{ XOR } C_{i-1})$, where $C_0 = IV = 0000$.

Block 1: C_1 = (1010 XOR 0000) XOR 1100 = 1010 XOR 1100 = 0110.

Block 2: C_2 = (1010 XOR 0110) XOR 1100 = 1100 XOR 1100 = 0000.

Block 3: C_3 = (1111 XOR 0000) XOR 1100 = 1111 XOR 1100 = 0011.

Block 4: C_4 = (1010 XOR 0011) XOR 1100 = 1001 XOR 1100 = 0101.

Therefore, CBC ciphertext = [0110, 0000, 0011, 0101].

(b) Comparison of ciphertext outputs

Block 1: plaintext 1010; ECB ciphertext 0110; CBC ciphertext 0110.

Block 2: plaintext 1010; ECB ciphertext 0110; CBC ciphertext 0000.

Block 3: plaintext 1111; ECB ciphertext 0011; CBC ciphertext 0011.

Block 4: plaintext 1010; ECB ciphertext 0110; CBC ciphertext 0101.

The ECB output repeats 0110 for every occurrence of plaintext block 1010. In CBC, the repeated plaintext blocks at positions 1, 2, and 4 produce different ciphertext blocks because each block depends on the previous ciphertext block.

(c) Which mode hides patterns better and why?

CBC hides repeated plaintext patterns better than ECB. ECB encrypts every block independently, so identical plaintext blocks encrypted with the same key produce identical ciphertext blocks. This can reveal patterns in the original data.

CBC combines each plaintext block with the preceding ciphertext block before encryption. Therefore, a block's ciphertext depends on both its plaintext and the previous ciphertext. A fresh, unpredictable IV also helps ensure that encrypting the same plaintext with the same key does not repeatedly produce the same ciphertext.

CBC does not provide authentication by itself and requires correct IV handling and padding. Modern applications commonly prefer authenticated encryption modes such as AES-GCM.

Conclusion: CBC provides better pattern concealment than ECB under the stated teaching assumption. The simplified XOR operation is not suitable for real security; a secure block cipher must be used.

3. DES vs 3-DES vs Blowfish Performance Study

Given:

a. DES key size = 56 bits, time = 2 ms/block

b. 3-DES key size = 168 bits, time = 6 ms/block

c. Blowfish key size = 128 bits, time = 1 ms/block

For 1000 blocks of data:

i.Calculate total encryption time for each algorithm

ii.Analyze trade-offs between security and performance

iii.Recommend the best algorithm for secure real-time communication

Answer 3: DES vs 3-DES vs Blowfish Performance Study

Given:

DES key size = 56 bits and time = 2 ms/block;

3-DES key size = 168 bits and time = 6 ms/block;

Blowfish key size = 128 bits and time = 1 ms/block.

Number of blocks = 1000.

(i) Calculate total encryption time for each algorithm

Formula: Total encryption time = Number of blocks × encryption time per block.

DES:

T = 1000 × 2 ms = 2000 ms = 2 seconds.

3-DES:

T = 1000 × 6 ms = 6000 ms = 6 seconds.

Blowfish:

T = 1000 × 1 ms = 1000 ms = 1 second.

Thus, for 1000 blocks, DES takes 2 seconds, 3-DES takes 6 seconds, and Blowfish takes 1 second, based on the supplied measurements.

Blowfish is twice as fast as DES and six times as fast as 3-DES in this comparison.

(ii) Trade-offs between security and performance

DES:

DES has an effective key size of 56 bits, which is vulnerable to brute-force attacks using modern computing capabilities. Its supplied processing time is 2 seconds for 1000 blocks. Although it is relatively fast, it is no longer considered secure for protecting sensitive information.

3-DES:

Triple DES applies the DES operation three times and therefore has a higher computational cost. The stated 168-bit key represents three 56-bit keys; it does not provide 168 bits of effective security. It takes 6 seconds for 1000 blocks in the given data, making it the slowest option. It is considered legacy technology and is not recommended for new general-purpose systems.

Blowfish:

Blowfish supports variable-length keys, including a 128-bit key, and takes 1 second for 1000 blocks in the given measurements. It is faster than the other two options in this dataset and is stronger than single DES when appropriately configured. However, its 64-bit block size creates limitations when encrypting large volumes of data.

The time figures are supplied values for this assignment and should not be treated as universal benchmark results. Actual performance depends on hardware, software implementation, and data handling.

(iii) Recommend the best algorithm for secure real-time communication

Among the three algorithms, Blowfish is the fastest according to the supplied measurements and is preferable if the choice is restricted to these three based on the stated timing and basic security comparison.

However, for a new real-time communication system, AES-GCM is the better practical recommendation. It provides efficient encryption and authentication, protecting confidentiality and helping detect tampering. It is widely supported by modern hardware and software.

Conclusion: Blowfish is fastest among the three given choices, but AES-GCM is more appropriate for modern secure real-time communication. DES is insecure against brute force, and 3-DES is slow and obsolete for new deployments.

4. Diffie-Hellman Key Exchange Analysis

Given:

a. p=23, g=5

b. User A private key = 6

c. User B private key = 15

d. Compute public keys for A and B

e. Compute the shared secret key

f. Demonstrate how a Man-in-the-Middle attacker could alter the exchange

g. Suggest a secure method to prevent the attack

Answer 4: Diffie–Hellman Key Exchange Analysis

Given:

prime p = 23, generator g = 5, User A private key a = 6, and User B private key b = 15.

Diffie–Hellman allows two users to establish a shared secret over a public channel without sending their private keys to each other.

(a) Compute the public keys for A and B

User A's public key is $A = g^a \bmod p = 5^6 \bmod 23$.

Since $5^2 \bmod 23 = 2$ and $5^4 \bmod 23 = 4$, $A = (5^4 \times 5^2) \bmod 23 = (4 \times 2) \bmod 23 = 8$.

User B's public key is $B = g^b \bmod p = 5^{15} \bmod 23$.

Repeated squaring gives $5^2 \bmod 23 = 2$, $5^4 \bmod 23 = 4$, and $5^8 \bmod 23 = 16$.

Since $15 = 8 + 4 + 2 + 1$, $B = (16 \times 4 \times 2 \times 5) \bmod 23 = 19$.

Therefore, User A's public key is 8 and User B's public key is 19.

(b) Compute the shared secret key

User A computes the shared secret using User B's public key:

$K_a = B^a \bmod p = 19^6 \bmod 23$. Since $19^2 \bmod 23 = 16$ and $19^4 \bmod 23 = 3$, $K_a = (3 \times 16) \bmod 23 = 2$.

User B computes the shared secret using User A's public key:

$K_\beta = A^b \bmod p = 8^{15} \bmod 23$. Repeated squaring gives $8^2 \bmod 23 = 18$, $8^4 \bmod 23 = 2$, and $8^8 \bmod 23 = 4$. Thus, $K_\beta = (4 \times 2 \times 18 \times 8) \bmod 23 = 2$.

Since $K_a = K_\beta = 2$, the shared secret key is K = 2.

Both users derive the same value without transmitting their private keys.

(f) Demonstrate a Man-in-the-Middle (MITM) attack

A Man-in-the-Middle attacker intercepts and replaces the public values exchanged between two users. Suppose the attacker chooses private value m = 3.

The attacker's public value is $X = g^m \bmod p = 5^3 \bmod 23 = 125 \bmod 23 = 10$.

The attacker intercepts A's public value 8 and sends X = 10 to User B.

The attacker intercepts B's public value 19 and sends X = 10 to User A.

Neither user can detect the substitution if the exchange is unauthenticated.

User A derives a secret with the attacker:

$K_AM = X^a \bmod 23 = 10^6 \bmod 23 = 6$.

The attacker independently calculates the same value from A's public key:

$8^3 \bmod 23 = 6$.

For the second connection, the attacker chooses a different private value $m_2 = 4$ and sends $X_2 = g^4 \bmod 23 = 5^4 \bmod 23 = 4$ to User B while pretending to be User A.

User B calculates $K_BM = X_2^b \bmod 23 = 4^{15} \bmod 23 = 3$.

The attacker calculates the same secret from B's public value: $B^4 \bmod 23 = 19^4 \bmod 23 = 3$.

The attacker therefore establishes two separate secrets: K_AM = 6 with User A and K_MB = 3 with User B. The attacker can decrypt a message received from A using the first secret, then re-

encrypt it using the second secret before forwarding it to B, and do the reverse for messages from B. Neither user detects the substitution if the exchange is unauthenticated.

A valid MITM attack works by establishing two separate key exchanges: one between A and the attacker, and one between the attacker and B. The attacker uses different private values for the two exchanges, sends the corresponding public values to each party, and computes each matching shared secret. The attacker can then decrypt traffic on one side and re-encrypt it for the other side.

(g) Suggest a secure method to prevent the attack

The main defence is to authenticate the exchanged public keys. Digital signatures can bind each Diffie–Hellman public value to the sender's identity, and the receiver verifies the signature using a trusted public key.

Digital certificates issued by a trusted Certificate Authority (CA) help confirm that a public key belongs to the claimed identity. Authenticated protocols such as TLS use authenticated key exchange to verify identities and establish secure sessions. Key confirmation can also help each party verify that both sides derived the same session key.

Conclusion: Diffie–Hellman establishes a shared secret but does not, by itself, authenticate the participants. Authentication using signatures, certificates, and a secure protocol prevents silent public-key substitution.

5. ECC vs RSA Comparison

Given:

a. RSA key size = 1024 bits

b. ECC key size = 160 bits (equivalent security)

c. Device processing capacity = limited (IoT device)

d. Analyze computational cost for both systems

e. Estimate which consumes less power and memory

f. Justify which cryptosystem is better for the given scenario with reasoning

Answer 5: ECC vs RSA Comparison

Given:

RSA key size = 1024 bits;

ECC key size = 160 bits as stated in the question

The target device is an IoT device with limited processing capacity.

RSA (Rivest–Shamir–Adleman) and ECC (Elliptic Curve Cryptography) are public-key cryptographic systems used in applications such as digital signatures and key establishment.

(d) Analyze the computational cost of both systems

RSA performs modular exponentiation on large integers. A typical RSA encryption operation is $C = M^e \bmod n$. RSA with a 1024-bit modulus performs arithmetic on relatively large integers, and private-key operations such as decryption and signing can be computationally expensive.

ECC uses operations on points defined over an elliptic curve. A common operation is scalar multiplication, $Q = kP$, where P is a curve point, k is a private scalar, and Q is the resulting public point. ECC generally achieves comparable security with much shorter keys, and many constrained implementations require less computation for comparable-security operations.

The precise cost depends on the operation, processor architecture, software or hardware implementation, and selected curve or RSA parameters.

Comparison: RSA uses modular exponentiation and larger key representations; ECC uses elliptic-curve point operations and smaller key representations. ECC generally reduces key storage and communication overhead. RSA private-key operations can be expensive, while ECC is often efficient for constrained devices when equivalent security levels are compared.

Parameter	**RSA**	**ECC**
Given key size	1024 bits	160 bits
Mathematical operation	Modular exponentiation	Elliptic-curve point operations
Processing requirement	Generally higher	Generally lower for comparable security
Memory requirement	Higher	Lower
Communication overhead	Larger keys and signatures	Smaller keys and signatures

(e) Estimate which consumes less power and memory

ECC generally consumes less memory because its keys and signatures are smaller than RSA keys and signatures at comparable security levels. Smaller keys also reduce the amount of data transmitted during authentication and key establishment.

RSA private-key operations can require substantial processing, which can increase energy consumption. ECC often performs comparable-security operations with less computational work in constrained implementations, making it attractive for battery-powered IoT devices.

The actual power difference depends on the cryptographic operation, processor architecture, implementation, and hardware acceleration. Therefore, ECC is generally expected to use less power and memory in the stated IoT scenario, although a specific device should be benchmarked.

(f) Justify which cryptosystem is better for the given scenario

ECC is generally the preferred approach for an IoT device with limited processing capacity, memory, bandwidth, and battery power, provided that a modern and appropriately secure curve is selected.

First, ECC uses smaller keys, reducing storage and transmission requirements. Second, it can provide comparable security with lower resource usage in many implementations. Third, smaller public keys and signatures reduce communication overhead and can make connection setup more efficient. Fourth, these advantages are valuable when many constrained devices need to authenticate or establish session keys.

Security qualification: the key sizes stated in the question are historical examples. A 160-bit ECC key does not provide equivalent security to a 1024-bit RSA key and is not considered secure for modern applications. For modern deployments, RSA-3072 and a suitable 256-bit elliptic-curve system are commonly used for roughly comparable security levels.

ECC does not automatically provide message encryption by itself in every use case. An appropriate protocol should be used for authenticated key exchange, followed by symmetric authenticated encryption such as AES-GCM.

Conclusion: ECC is generally better suited to the given resource-constrained IoT scenario because it offers compact keys and efficient operations. A modern curve such as NIST P-256 or a suitable Curve25519-based protocol should be used rather than the outdated 160-bit example.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation